

Course:	INFO3142
Professor:	Jim Cooper
Project:	Project #1 – Abstraction Reasoning Corpus (ARC) Version 1.0
Due Dates:	Section 01 - Monday, October 18, 2025 11:30 pm Section 02 - Wednesday, October 20, 2025 11:30 pm Section 03 - Thursday, October 21, 2025 11:30 pm
Submitting:	Please see the last page for instructions

How will my project be marked?

- This project counts for 20% of your final grade and will be evaluated using the following grid:

Marks Available	What are the Marks Awarded For?	Mark Assigned
10	Creation of 3 ARC puzzle solutions – done by the student using a “concept” based approach (see requirements below). In addition, each of the HTML page solutions must run correctly. Code comments are required.	
6	For each of the 3 hand-crafted solutions, a GEN-AI version of the same ARC puzzle is to be created using the LLM of your choice. Important: The original ARC-AGI JSON files for your selected puzzles must be included with your submission: Example: 00d62c1b.json	
2	All submitted solutions work perfectly.	
2	Proper zipped submission.	
20	Total	

Project Description

The main goal of this project is to work with Francois Chollet’s Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) in an effort to understand some of what might be required to reach AGI (Artificial General Intelligence).

ARC-AGI is sometimes called the IQ test for LLMs. This software and accompanying data, is just a series of visual puzzles, which have been designed to resist the typical LLM transformer “saturation” training.

Since each puzzle is intended to be a unique problem-solving experience, LLMs initially didn’t do very well solving the puzzles. In contrast human beings are very good at solving ARC puzzles as you’ll see when you try some yourself. In an effort to get AI developers interested in ARC-AGI, an AI company (Zapier) created a [competition](#) for the most interesting results and awarded a million dollars in total prize money. In the end, ARC-1 proved difficult for LLMs to solve although

not impossible and by the time the contest ended, most state-of-the-art LLMs could solve a reasonable number of puzzles although not as many as the average human.

This all happened in 2024 and since then, a new version of ARC-AGI has been created which is even better at resisting LLM memorization.

As a result, there are now two versions of the software (ARC-AGI-1 and ARC-AGI-2).

We're mostly interested in the first version because it will be more manageable for a course project. Before reading any further, please go to the following GitHub link:

[ARC-AGI-1 Github Site](#)

- Read about ARC-1 puzzles
- Download and unzip the software and JSON data
- Try solving a few of the puzzles yourself

As you'll see from the project requirements below, we want to develop an appreciation for the difficulty of coding solutions for ARC puzzles from the perspective of concepts vs raw data output.

Specific Project Requirements:

1. For each of the puzzles you've solved, try to identify the series of steps required to reach a correct solution, i.e. a specific algorithm for the current puzzle. Note that some puzzles are more difficult than others.
2. For instance, the example solution included with this project spec requires an understanding of the conceptual idea of a container within which we can render a colour fill. See figure-1 in the Notes section below and try the HTML page yourself.
3. Have a look at the GPT-5 version, also included.
4. What we want you to do for this project is to find three puzzles in either ARC-1 or ARC-2 (or some combination) that you can solve yourself. Once you've identified the necessary concepts for each solution, create a set of separate HTML pages (including JavaScript and CSS) that renders your solutions.
5. For each of your ARC puzzles, we also want you to include a solution that's been created by the LLM of your choice. Make sure you test this AI generated version because some of the LLMs are not as good at solving ARC puzzles as others. You should take a screenshot

Notes:

[It's Not About Scale, It's About Abstraction](#)

[How We Get to AGI](#)

[On the Measure of Intelligence](#)

Simplified overview of the previous Francois Chollet paper:

Francois Chollet argues that intelligence should be measured not by task performance alone, but by how efficiently an agent (like a human or AI system) can adapt to new tasks using limited prior knowledge and experience.

Key Points of Chollet's Paper:

1. Intelligence is about generalization:
 - A truly intelligent agent can adapt to new problems quickly; not just perform well on tasks it has seen before.
2. Problems with current AI benchmarks:
 - Many benchmarks reward memorization or brute-force training, not real understanding.
 - For example, large AI models may perform well because they've seen similar data before—not because they are “smart.”
3. New measure of intelligence:
 - Chollet suggests evaluating agents based on:
Skill: how well they solve tasks.
Experience: how much prior knowledge they need.
Generalization difficulty: how novel or hard the task is compared to what they've seen.
4. Efficiency is key:
 - **Intelligence = how efficiently an agent can acquire new skills in unfamiliar situations, using limited resources.**

Summary of Paper:

The paper shifts the focus from raw performance to adaptive skill-learning, offering a more meaningful way to assess both human and machine intelligence.

Examples:

Figure-1 (Concept based solution as provided)

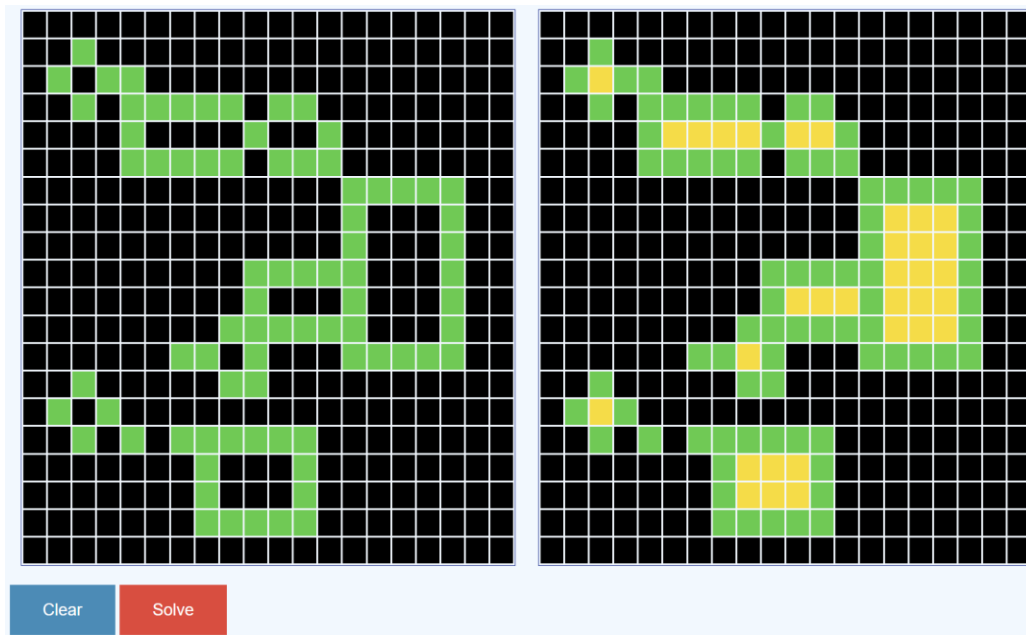


Figure-2 (ARC1 - Example demonstrations and design canvas – from GitHub)



Figure-3 (ARC2 - Example demonstrations and design canvas – from GitHub)

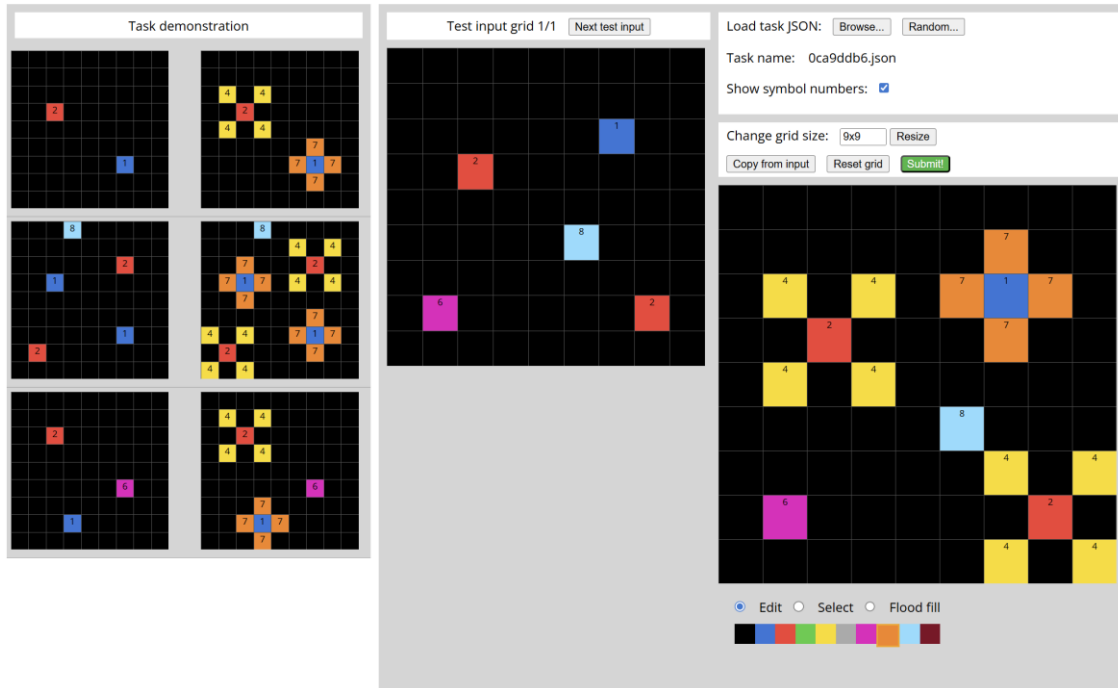
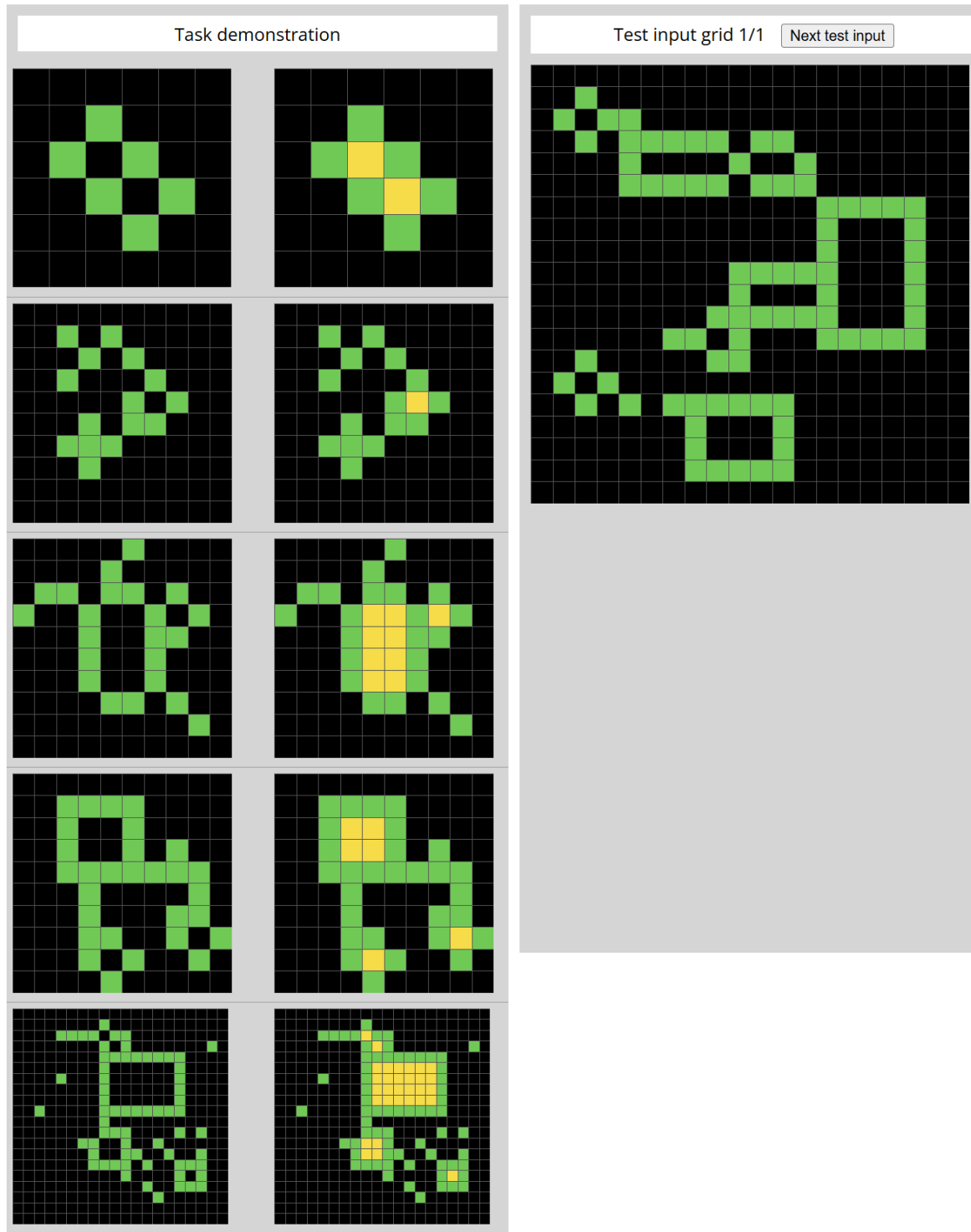


Figure-4 (modified – JSON converted to JS external data file)

```
1 var obj = {"train": [{"input": [[0, 0, 0, 0, 0, 0], [0, 0, 3, 0, 0, 0], [0, 3, 0, 3  
    , 0, 0], [0, 0, 3, 0, 3, 0], [0, 0, 0, 3, 0, 0], [0, 0, 0, 0, 0, 0]], "output": [[0  
    , 0, 0, 0, 0, 0], [0, 0, 3, 0, 0, 0], [0, 3, 4, 3, 0, 0], [0, 0, 3, 4, 3, 0], [0, 0  
    , 0, 3, 0, 0], [0, 0, 0, 0, 0, 0]]}, {"input": [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0,  
    0, 3, 0, 3, 0, 0, 0, 0, 0, 0], [0, 0, 0, 3, 0, 3, 0, 0, 0, 0], [0, 0, 3, 0, 0, 3, 0,  
    0, 0, 0], [0, 0, 0, 0, 0, 3, 0, 3, 0, 0], [0, 0, 0, 3, 0, 3, 3, 0, 0, 0], [0, 0, 3,  
    3, 3, 0, 0, 0, 0, 0], [0, 0, 0, 3, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
    0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]], "output": [[0, 0, 0, 0, 0, 0, 0, 0], [0, 0,  
    3, 0, 3, 0, 0, 0, 0, 0], [0, 0, 0, 3, 0, 3, 0, 0, 0], [0, 0, 3, 0, 0, 3, 0, 3,  
    0, 0, 0], [0, 0, 0, 0, 0, 3, 4, 3, 0, 0], [0, 0, 0, 3, 0, 3, 3, 0, 0, 0], [0, 0, 3,  
    3, 3, 0, 0, 0, 0, 0, 0], [0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0,  
    0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]}], {"input": [[0, 0, 0, 0, 3, 0, 0, 0], [0  
    , 0, 0, 0, 3, 0, 0, 0, 0, 0], [0, 3, 3, 0, 3, 3, 0, 3, 0, 0], [3, 0, 0, 3, 0, 0, 3,  
    0, 3, 0], [0, 0, 0, 3, 0, 0, 3, 3, 0, 0], [0, 0, 0, 3, 0, 0, 3, 0, 0, 0], [0, 0, 0  
    , 3, 0, 0, 3, 0, 0, 0], [0, 0, 0, 0, 3, 3, 0, 3, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 3,  
    0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]], "output": [[0, 0, 0, 0, 0, 3, 0, 0, 0, 0], [0  
    , 0, 0, 0, 3, 0, 0, 0, 0, 0], [0, 3, 3, 0, 3, 0, 3, 0, 0], [3, 0, 0, 3, 4, 4, 3, 0  
    , 4, 3, 0], [0, 0, 0, 3, 4, 4, 3, 3, 0, 0], [0, 0, 0, 3, 4, 4, 3, 0, 0, 0], [0, 0, 0  
    , 3, 4, 4, 3, 0, 0, 0], [0, 0, 0, 0, 3, 3, 0, 3, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 3,  
    0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]}], {"input": [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0  
    , 0, 3, 3, 3, 3, 0, 0, 0], [0, 0, 3, 0, 0, 3, 0, 0, 0], [0, 0, 3, 0, 0, 3, 0, 0, 3, 0  
    , 3, 0, 0], [0, 0, 3, 3, 3, 3, 3, 3, 3, 0, 0], [0, 0, 0, 3, 0, 0, 0, 0, 3, 0, 0], [0, 0,  
    0, 3, 0, 0, 0, 3, 3, 0], [0, 0, 0, 3, 3, 0, 0, 3, 0, 3], [0, 0, 0, 3, 0, 3, 0, 0, 3  
    , 0], [0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0]], "output": [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0  
    , 0, 3, 3, 3, 3, 0, 0, 0], [0, 0, 3, 4, 4, 3, 0, 0, 0], [0, 0, 3, 4, 4, 3, 0, 3, 0  
    , 3, 0, 0], [0, 0, 3, 3, 3, 3, 3, 3, 3, 0, 0], [0, 0, 3, 0, 0, 0, 0, 3, 0, 0], [0, 0,  
    0, 3, 0, 0, 0, 3, 3, 0], [0, 0, 0, 3, 3, 0, 0, 3, 4, 3], [0, 0, 0, 3, 4, 3, 0, 0, 3  
    , 0], [0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0]}], {"input": [[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0  
    , 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0  
    , 0, 0], [0, 0, 0, 0, 0, 3, 3, 3, 3, 0, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0,
```

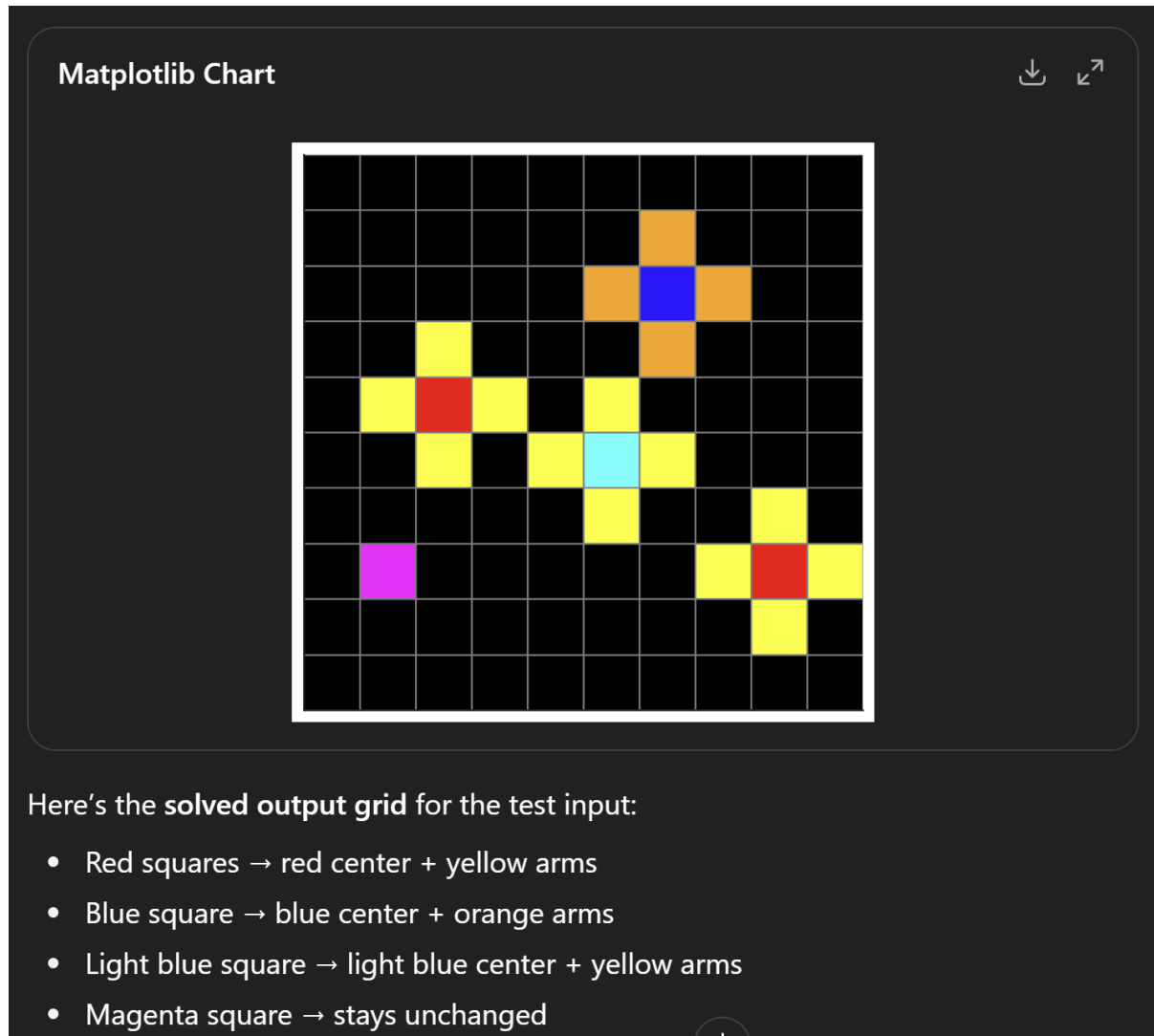
Figure-5 (The kind of input you give your LLM)



Task demonstration

Test input grid 1/1

Figure-6 (GPT-5 Conceptual Solution)



How should I submit my project?

Electronic Submission:

Submit your program files to the *Info3142 "Project 1"* electronic submission folder in *FanshaweOnline*. These files should be submitted as a single "zip" file containing your web application's complete website.

I strongly recommend that you test your own submission to ensure that nothing has been missed.

Submit your project on time!

Project submissions must be made on time! Late projects will be subject to divisional policy on missed tests and late projects. In accordance with this policy, no late projects will be accepted without prior notification being received by the instructor from the student.

Submit your own work!

It is considered cheating to submit work done by another student or from another source. Helping another student cheat by sharing your work with them is also not tolerated. Students are encouraged to share ideas and to work together on practice exercises, but any code or documentation prepared for a project must be done by the individual student. Penalties for cheating or helping another student cheat may include being assigned zero on the project with even more severe penalties if you are caught cheating more than once. Just submit your own work and benefit from having made the effort on your own.